

Design and Evaluation of a Spreadsheet-Driven Static Portfolio Deployment Pipeline Using Google Sheets and GitHub Actions

Mohan Manideep
Personal Portfolio Website Project
mohanmanideep.github.io

June 25, 2026

Abstract

Static portfolio websites are commonly used to present professional profiles, academic backgrounds, technical skills, and project work. However, maintaining such websites often requires repeated source-code modifications for simple content updates, creating unnecessary maintenance overhead and increasing the risk of introducing front-end errors. This paper presents the design and evaluation of a spreadsheet-driven static portfolio deployment pipeline that separates portfolio content management from front-end source code. The implemented system uses Google Sheets as a structured content source, Google Apps Script as an update trigger, GitHub Actions as an automated continuous deployment pipeline, React and Vite for front-end generation, and GitHub Pages for static hosting. Portfolio data is fetched from spreadsheet tabs, transformed into a JSON data file, consumed by the React front-end, and deployed automatically after a repository dispatch event. The system was evaluated using deployment measurements, Lighthouse auditing, and browser runtime observations. Across six successful deployment runs, the workflow achieved an average total duration of 33.0 seconds and an average deployment job duration of 23.8 seconds. A private-window Lighthouse audit reported scores of 91 for Performance, 100 for Accessibility, 100 for Best Practices, and 91 for SEO. The results show that a spreadsheet-driven workflow can provide a lightweight, low-cost, and maintainable alternative to manually edited static portfolio websites while preserving acceptable front-end performance and deployment reliability.

Keywords: Spreadsheet-driven CMS, Static site deployment, Google Sheets, GitHub Actions, GitHub Pages, React, Vite, Portfolio website, Deployment automation, Front-end performance

1 Introduction

Static portfolio websites are easy to host and maintain, but their content is often stored directly inside frontend

source files. This creates a problem when the user wants to update projects, skills, experience, or profile details, because even small content changes may require editing code, rebuilding the site, and redeploying it.

This paper presents a spreadsheet-driven portfolio deployment pipeline that separates content management from frontend development. The system uses Google Sheets as a lightweight content source, Google Apps Script as an update trigger, GitHub Actions for automation, React and Vite for the frontend, and GitHub Pages for hosting [5, 6, 3, 4, 1, 2].

In the implemented system, the user updates portfolio data in spreadsheet tabs instead of editing React components. A deployment workflow fetches the latest spreadsheet data, converts it into `portfolio.json`, builds the frontend, and deploys the updated website automatically.

The main contribution of this work is a low-cost and repeatable workflow for managing a static portfolio website through a spreadsheet-based content pipeline. The system is evaluated using GitHub Actions deployment times, Lighthouse audit results, and browser runtime observations [7, 8].

2 Problem Definition

In a conventional static portfolio website, content and presentation are often stored together in the frontend codebase. This means that updating portfolio information, such as projects, skills, education, or experience, requires modifying source files and redeploying the website.

This approach creates three main problems:

- **Content-code coupling:** Portfolio data is mixed with frontend logic and layout.
- **Manual update effort:** Every content change requires code editing, rebuilding, and deployment.
- **Risk of accidental errors:** Simple content updates may unintentionally affect the website structure or design.

The problem addressed in this project is how to manage and deploy frequently changing portfolio content without directly modifying frontend source code for every update.

3 Objectives

The objectives of this project are:

- To separate portfolio content from frontend source code.
- To use Google Sheets as a structured content source for profile, education, experience, projects, skills, links, and media.
- To automate the content update and deployment process using Google Apps Script and GitHub Actions.
- To generate a static React/Vite website from spreadsheet-based data.
- To deploy the website through GitHub Pages with no direct hosting cost.
- To evaluate the system using deployment duration, artifact size, Lighthouse scores, and browser run-time observations.

4 Proposed System

The proposed system is a spreadsheet-driven static portfolio website. Instead of storing portfolio content directly inside React components, the content is maintained in Google Sheets. Each spreadsheet tab represents a separate content category, such as profile, links, education, experience, projects, skills, interests, media, and site configuration.

The update process starts from the spreadsheet. When the user clicks the update button, Google Apps Script sends a request to GitHub and triggers a `repository_dispatch` event. GitHub Actions then runs the deployment workflow, fetches the latest spreadsheet data, generates `portfolio.json`, builds the React/Vite frontend, and deploys the final static website to GitHub Pages.

This system separates three responsibilities: Google Sheets manages content, React/Vite manages presentation, and GitHub Actions manages automation and deployment. This separation reduces manual code editing and makes portfolio updates easier to perform.

5 System Architecture

The system follows a data-driven deployment architecture. Portfolio content is stored in Google Sheets and automatically converted into JSON during deployment.

The frontend application reads the generated JSON file and renders the website dynamically.



Figure 1: System architecture of the automated portfolio website deployment pipeline.

6 Implementation

The implementation consists of four main components: spreadsheet-based content storage, update triggering, automated data generation, and frontend rendering.

6.1 Spreadsheet-Based Content Storage

Google Sheets is used as the content source for the portfolio website. The spreadsheet is organised into multiple tabs, including Profile, Links, Education, Experience, Projects, Skills, Interests, Media, and Site_config. Each tab represents one content category and stores data in a structured row-column format.

This structure allows content to be updated without modifying the React source files. For example, adding a new project requires adding a row in the Projects tab instead of editing frontend components.

6.2 Update Trigger Using Google Apps Script

Google Apps Script is used to connect the spreadsheet with GitHub. An update button has been added inside the spreadsheet. When clicked, the script sends a request to the GitHub API and triggers a `repository_dispatch` event.

This event-based trigger allows the deployment workflow to start from the content management interface itself. Therefore, the user can update the spreadsheet and redeploy the website without opening the GitHub repository manually.

6.3 Automated Data Generation

After the `repository_dispatch` event is received, GitHub Actions starts the deployment workflow. The workflow executes the `fetch-sheet-data.js` script, which fetches data from the Google Sheet and converts it into a structured JSON file named `portfolio.json`.

The generated JSON file acts as the bridge between the spreadsheet and the frontend application. It standardises the portfolio content into a format that can be directly consumed by React components.

6.4 Frontend Rendering

The frontend is implemented using React and Vite. React components read data from `portfolio.json` and render the portfolio sections dynamically. These sections include profile information, education, experience, projects, skills, interests, links, and contact information.

Vite is used for development and production build generation. After the build step is completed, the generated static files are deployed to GitHub Pages.

6.5 Deployment to GitHub Pages

GitHub Pages hosts the final static website. Once the GitHub Actions workflow completes the build process, the deployment step publishes the generated site to the live portfolio URL. The live website is available at <https://mohanmanideep.github.io/> [9], and the project source code is maintained in the GitHub repository [10].

7 Evaluation and Results

The system was evaluated using deployment performance, Lighthouse frontend audit results, and runtime stability observation. Lighthouse was used to measure web performance, accessibility, best practices, and SEO [7]. Core Web Vitals were considered for interpreting frontend performance metrics such as Largest Contentful Paint, Cumulative Layout Shift, and Interaction to Next Paint [8].

7.1 Deployment Performance

Six successful GitHub Actions workflow runs were recorded. The results are shown in Table 1.

Table 1: GitHub Actions deployment performance across successful runs

Trial	Trigger	Total	Deploy	Size
1	<code>repository_dispatch</code>	39 s	27 s	117 KB
2	<code>repository_dispatch</code>	33 s	26 s	117 KB
3	<code>repository_dispatch</code>	32 s	21 s	116 KB
4	<code>push</code>	35 s	24 s	117 KB
5	<code>push</code>	34 s	23 s	117 KB
6	<code>push</code>	25 s	22 s	117 KB
Avg.	—	33.0 s	23.8 s	116.8 KB

Across six successful deployment trials, the automated GitHub Actions workflow completed with an average total duration of 33.0 seconds. The deployment job required an average of 23.8 seconds, and the generated GitHub Pages artifact size remained approximately 117 KB.

7.2 Frontend Performance

The website was evaluated using Google Lighthouse in a private browsing window to reduce browser-extension interference. The results are shown in Table 2.

Table 2: Lighthouse performance results in private browsing mode

Metric	Value
Performance Score	91
Accessibility Score	100
Best Practices Score	100
SEO Score	91
First Contentful Paint	1.4 s
Largest Contentful Paint	3.5 s
Total Blocking Time	40 ms
Cumulative Layout Shift	0.00
Speed Index	1.4 s

The Lighthouse audit indicates that the portfolio website achieved strong frontend quality results. The site received a Performance score of 91, Accessibility score of 100, Best Practices score of 100, and SEO score of 91. The Cumulative Layout Shift value of 0.00 indicates excellent visual stability during page loading.

7.3 Runtime Stability

A manual interaction stability observation was conducted by repeatedly interacting with buttons on the website while monitoring browser performance. The results are shown in Table 3.

Table 3: Runtime stability observation during repeated user interactions

Metric	Value
CPU Usage	3.4%
JavaScript Heap Size	11.5 MB
DOM Nodes	1,074

The runtime observation showed low CPU usage, low JavaScript heap size, and a moderate DOM node count during repeated user interactions. This suggests that the frontend remains lightweight and responsive during normal interactive use.

8 Results and Discussion

The evaluation results show that the proposed spreadsheet-driven deployment pipeline is practical for

a personal portfolio website. The average total deployment duration was 33.0 seconds, which is acceptable for a content update workflow where updates are not required in real time. The stable artifact size of approximately 117 KB also indicates that the generated static site remains lightweight.

The Lighthouse results show that the frontend maintains good quality after being generated from spreadsheet-based content. The website achieved a Performance score of 91, Accessibility score of 100, Best Practices score of 100, and SEO score of 91. These results suggest that separating content from source code did not negatively affect the quality of the deployed website.

The runtime observation also supports the lightweight nature of the frontend. During repeated user interactions, CPU usage remained low at 3.4%, and JavaScript heap size was 11.5 MB. This indicates that the React frontend can handle normal portfolio interactions without significant runtime overhead.

The main benefit of the system is maintainability. Content updates can be performed in Google Sheets, while the frontend code remains unchanged. This reduces the need for manual code editing and lowers the chance of introducing layout or logic errors during content updates. Therefore, the system is suitable for individual users who need a simple portfolio management workflow without using a full CMS platform.

9 Limitations

Although the system is effective for a personal portfolio website, it has some limitations. The system depends on the availability of Google Sheets and GitHub Actions. Since Google Sheets is used as the content source, incorrect spreadsheet formatting can affect the generated JSON data. GitHub Actions runtime changes may also require future workflow maintenance. In addition, images served from external sources such as Google Drive may affect Largest Contentful Paint if they are not optimised.

The system is not a full traditional CMS with user authentication, content previews, or role-based content editing. It is designed as a lightweight CMS workflow suitable for an individual portfolio.

10 Future Work

Future work can improve both the content pipeline and frontend performance. First, spreadsheet data validation can be added before deployment to detect missing fields, incorrect column names, or invalid URLs. This would reduce the risk of generating incomplete or incorrect `portfolio.json` files.

Second, image optimisation can be improved. Since media assets can affect Largest Contentful Paint, future versions can use compressed images, local image

hosting, responsive image sizes, and lazy loading for non-critical media.

Third, the update workflow can be extended with better error reporting. For example, if data fetching or deployment fails, the system could send a notification to the user through email or the spreadsheet interface.

Fourth, the portfolio interface can be enhanced with search, filtering, project categories, analytics, and automated broken-link checks. These features would make the website more useful as the amount of portfolio content grows.

Finally, the system can be generalised into a reusable template so that other users can create their own spreadsheet-driven portfolio websites with minimal setup.

11 Conclusion

This paper presented a spreadsheet-driven static portfolio deployment pipeline that separates portfolio content management from frontend source code. The implemented system uses Google Sheets for structured content storage, Google Apps Script for triggering updates, GitHub Actions for deployment automation, React and Vite for frontend generation, and GitHub Pages for static hosting.

The system allows portfolio updates to be performed through spreadsheet edits rather than direct source-code modifications. This improves maintainability and reduces the risk of introducing frontend errors during routine content updates.

The evaluation showed that the deployment workflow completed successfully across six measured runs with an average total duration of 33.0 seconds and an average deploy job duration of 23.8 seconds. Lighthouse testing in a private browsing window reported scores of 91 for Performance, 100 for Accessibility, 100 for Best Practices, and 91 for SEO. Runtime observation also showed low CPU and memory usage during repeated interactions.

Overall, the results indicate that a spreadsheet-driven content pipeline can be a practical, low-cost, and lightweight approach for managing a personal portfolio website. The system is not intended to replace a full CMS, but it provides an effective solution for individual users who need structured content management and automated static-site deployment.

References

- [1] React, “React documentation.” Available: <https://react.dev/>
- [2] Vite, “Vite documentation.” Available: <https://vite.dev/>
- [3] GitHub, “GitHub Actions documentation.” Available: <https://docs.github.com/actions>

- [4] GitHub, “GitHub Pages documentation.” Available: <https://docs.github.com/pages>
- [5] Google for Developers, “Google Sheets API documentation.” Available: <https://developers.google.com/sheets/api>
- [6] Google for Developers, “Google Apps Script documentation.” Available: <https://developers.google.com/apps-script>
- [7] Chrome for Developers, “Lighthouse documentation.” Available: <https://developer.chrome.com/docs/lighthouse>
- [8] web.dev, “Core Web Vitals.” Available: <https://web.dev/vitals/>
- [9] M. Manideep, “Personal portfolio website.” Available: <https://mohanmanideep.github.io/>
- [10] M. Manideep, “Portfolio website repository.” Available: <https://github.com/mohanmanideep/mohanmanideep.github.io>